UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026). Generated answers show working. Real-bank questions carry a labelled unofficial AI model answer, not an official scheme.

2. Marking points show the answer and mark allocation. Check model-generated real-bank points against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Generated slots: 2. Real-bank slots: 9. Unofficial model answers:

   9/9.
   For local study use only; model answers are unofficial.

5. Structure from 20 unique CSC429 past papers; every real question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 10 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION A1 (5 marks)

Topic: Computer generations and history

Source: CSC429 (6).pdf, page(s) 2

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Machine X is the Jacquard Loom (Jacquard weaving machine / Jacquard loom), developed by Joseph Marie Jacquard.

(b) The Jacquard Loom was developed in the pre-computer / mechanical era, conventionally placed in the "Generation 0" (pre-first-generation) period — i.e. before the first computer generation (vacuum-tube era). It dates from 1801, so it is not part of the electronic computer generations (first to fifth); it is regarded as a forerunner of computer generations.

(c) The deck of punched cards functions as stored program/instructions and data, so the equivalent computer component is memory (main store / program store), specifically the stored-program memory that holds the program (instructions) and data. (The punched cards are also read by a card reader, but the component with the same function — holding the control information — is memory.)

(d) Input: the deck of punched cards (the punched-card pattern/instructions) fed into the machine, together with the warp threads/yarn. Output: the woven fabric with the intricate pattern (the patterned textile/cloth).

Unofficial marking guide:

- *1 mark: (a) correctly names Machine X as the Jacquard Loom (Jacquard weaving machine).*
- *1 mark: (b) states the correct generation — pre-computer/Generation 0 (before first generation, mechanical era, c. 1801).*
- *1 mark: (c) names memory (stored-program memory) as the component with the same function as the punched-card deck.*
- *2 marks: (d) 1 mark for correct input (punched cards / card deck, with yarn/thread), 1 mark for correct output (patterned woven fabric/cloth).*

Model confidence: medium.

Model note: The placard text and Figure 1 are only partially available; the identification of Machine X as the Jacquard Loom is inferred from the placard wording (Joseph Marie Jacquard, punched cards, textile mills). Generation answer assumes the lecturer treats the Jacquard Loom as a pre-computer/Generation 0 forerunner rather than a numbered electronic generation; if the course counts it differently, adjust (b). For (c), 'memory' is the intended stored-program analogy; a card reader could be argued but is the reading device, not the stored information.

QUESTION A2 (5 marks)

Topic: Parallel and multiprocessor architecture

Source: CSC429 (5).pdf, page(s) 6

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

The question asks for completion of a taxonomy of parallel processor architecture with appropriate examples. The standard taxonomy (Flynn-style / shared-memory vs message-passing) is filled in as follows, with one example per category:

[Taxonomy completion] Parallel processor architectures divide into two main branches: (i) Shared-memory multiprocessors (tightly coupled), where all processors access a common memory, and (ii) Message-passing multicomputers (loosely coupled), where each processor has its own private memory and processors communicate by passing messages over an interconnection network.

[Shared-memory branch] Shared-memory multiprocessors subdivide into: (a) Uniform Memory Access (UMA) — all processors have equal access time to all memory modules, e.g. a symmetric multiprocessor (SMP) such as a dual/quad-core Intel Xeon server; (b) Non-Uniform Memory Access (NUMA) — memory access time depends on which memory module is addressed, e.g. a large-scale cache-coherent NUMA (ccNUMA) server such as SGI Origin; (c) Cache-Only Memory Architecture (COMA) — memory is treated as a pool of caches, e.g. the Kendall Square Research KSR-1.

[Message-passing branch] Message-passing multicomputers subdivide into: (a) Massively Parallel Processors (MPP) — many processors with private memories connected by a dedicated high-speed network, e.g. Intel Paragon or Cray T3D; (b) Clusters / Workstations-of-workstations (COWs) — commodity computers connected by a LAN, e.g. a Beowulf cluster; (c) Distributed shared-memory

© Hak Cipta Universiti Teknologi MARA

systems built on message passing, e.g. a cluster running a DSM/software DSM layer.

[Example requirement] Each leaf of the taxonomy must be paired with a named, real example as shown above; the example should match the defining property of that category (equal access time for UMA, private memory plus network for MPP, commodity nodes for clusters).

Unofficial marking guide:

• 1 mark: correct top-level division into shared-memory (tightly coupled) and message-passing (loosely coupled) branches

• 1 mark: correct sub-categories under shared memory (UMA / NUMA / COMA)

• 1 mark: correct sub-categories under message passing (MPP / cluster / COW)

• 1 mark: a valid, named example for each shared-memory sub-category

• 1 mark: a valid, named example for each message-passing sub-category

Model confidence: medium.

Model note: The taxonomy figure (CSC429-2016-06-D1) is only available as extracted text, so the exact branch labels and number of leaves in the original diagram cannot be verified. The answer assumes the standard Flynn/shared-vs-message-passing taxonomy used in CSC429 notes; if the figure uses different labels (e.g. SIMD/MIMD or UMA/NUMA/COMA only), map the examples accordingly. Examples are illustrative and any correct named system should be accepted.

QUESTION A3 (5 marks)

Topic: Instruction formats and addressing modes

Source: CSC429 (1).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The technique is direct addressing (also called absolute addressing). The instruction's address field A contains the actual/effective address of the operand, so the operand is fetched directly from memory location A — the figure shows the address field pointing straight into memory to the operand, with no intermediate pointer.

(b) How it is used: the instruction is fetched and decoded; the CPU takes the address field A of the instruction and places it on the address bus as the effective address (EA = A); memory is read at that location and the operand is transferred to the CPU (e.g. into a register or the ALU). No further address calculation or extra memory access for an address is needed — one operand fetch from memory at address A. Example: LOAD 500 fetches the contents of memory location 500 into the accumulator.

(c) Advantage: simple and fast — only one memory reference is needed to get the operand (no extra memory access to compute an address), and no address-calculation hardware is required. Disadvantage: limited addressable range (the address field must be large enough to hold a full memory address, making instructions long) and it is inflexible — the address is fixed at assembly time, so it cannot easily support relocation, arrays/pointers or dynamic data structures.

Unofficial marking guide:

• 1 mark: correctly names the technique as direct addressing (absolute addressing).

• 1 mark: states that the address field A is the effective address of the operand (EA = A).

• 1 mark: explains the fetch sequence — instruction decoded, address A sent to memory, operand read directly from location A into the CPU.

• 1 mark: gives one valid advantage (e.g. simple/fast, only one memory access, no address calculation).

• 1 mark: gives one valid disadvantage (e.g. limited address range/long instruction format, inflexible/fixed address, poor support for relocation or data structures).

Model confidence: high.

Model note: Figure 2 is only available as extracted text; the answer assumes the diagram shows the instruction's address field A pointing directly to the operand in memory (the standard direct-addressing depiction). If the figure instead shows A pointing to a memory word that itself holds the operand's address, the technique would be indirect addressing — this must be confirmed from the figure.

QUESTION A4 (5 marks)

Topic: Data representation and number systems

Answer: -20.12510 = 0xC1A10000 (sign=1, exponent=10000011, fraction=01000010000000000000000)

• 1 mark for the sign bit.

• 1 mark for the biased exponent.

• 1 mark for the 23-bit fraction.

• 1 mark for the correct 32-bit grouping.

• 1 mark for the final hexadecimal value.

© Hak Cipta Universiti Teknologi MARA

QUESTION A5 (10 marks)

Topic: Memory hierarchy and cache

Source: CSC429 (4).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

Access methods based on hardware implementation of the storage device

1. Sequential access. Data is stored as a linear sequence of records, and to reach a particular record the device must read through all preceding records in order; access time therefore depends on the position of the data and is highly variable. Access is strictly in the order in which the data is physically arranged. *Example:* magnetic tape (also punched paper tape).

2. Direct access. Each block or record has a unique address based on its physical location, so the device can jump close to the required block and then search sequentially within it; access time is still variable but much less order-dependent than pure sequential access. *Example:* magnetic disk / hard disk drive (also floppy disk, CD/DVD-ROM).

3. Random access. Any storage location can be selected and accessed directly in essentially the same, constant time, independent of the address or of previous accesses; each location has a unique, electronically addressable identity. *Example:* main memory (RAM / semiconductor memory, e.g. DRAM).

4. Associative access. Data is not addressed by location but retrieved by matching its content (a key or tag) against stored entries in parallel; the hardware compares the search key against all words simultaneously and returns the matching word(s). *Example:* cache memory (associative/fully-associative cache) or a content-addressable memory (CAM) / associative memory.

(If a fourth distinct category is preferred, read-only/ROM-type storage may be cited, but the standard four access methods are sequential, direct, random and associative.)

Unofficial marking guide:

- *1 mark: correctly naming/describing sequential access (data read in order, access time depends on position)*
- *1 mark: appropriate example for sequential access (magnetic tape)*
- *1 mark: correctly naming/describing direct access (addressable blocks, direct jump then sequential search)*
- *1 mark: appropriate example for direct access (magnetic disk/HDD)*
- *1 mark: correctly naming/describing random access (any location accessed in constant time, addressable)*
- *1 mark: appropriate example for random access (main memory/RAM)*
- *1 mark: correctly naming/describing associative access (content/key match, parallel comparison)*
- *1 mark: appropriate example for associative access (cache/CAM)*
- *2 marks: overall accuracy and clarity (one mark per two well-explained methods, or 2 marks for four fully correct method+example pairs)*

Model confidence: high.

Model note: Question text is clear; no OCR correction needed. The standard four access methods (sequential, direct, random, associative) are assumed. Some textbooks also list 'read-only' as a fifth category; the answer notes this but marks the standard four.

QUESTION A6 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (16).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The two machine-cycle processes governed by main memory are the Fetch and Memory (data transfer) / Indirect processes — i.e., the two phases in which the CPU must access main memory. Fetch: the CPU reads the next instruction from main memory. The address in the PC is placed on the address bus (via the MAR), the memory is read, and the instruction word is transferred over the data bus into the MBR and then to the IR; the PC is incremented. Main memory governs this because it supplies the instruction word and determines the access time. Memory/Indirect (data) process: when an instruction uses memory operands, the CPU reads (or writes) data from/to main memory. The effective address is placed in the MAR, memory is accessed, and the data word passes through the MBR to/from a register. Main memory governs this because it holds the operand data and controls read/write timing. (In the standard machine cycle these are the two memory-referencing steps; the Execute phase itself is CPU-internal unless it performs a memory read/write.)

(b) Two operations comprised in the Execute phase (any two): data transfer between registers (e.g.,

© Hak Cipta Universiti Teknologi MARA

MOV/register-to-register), arithmetic/logic operations (ADD, SUB, AND, OR, NOT, shift), control transfer / branch (jump, conditional branch), and I/O or memory read/write operations. (List any two.)

(c) During the Fetch cycle: the PC holds the address of the next instruction. That address is copied to the MAR and placed on the address bus; memory returns the instruction word, which is loaded into the MBR and then into the IR. The PC is then incremented (PC ← PC + 1, or + instruction length) so it points to the following instruction. The IR now holds the fetched instruction opcode, which is passed to the control unit/decoder to determine the operations for the Execute phase. Thus the PC supplies the address and advances, while the IR latches and holds the instruction just fetched.

Unofficial marking guide:

- *1 mark: identifies Fetch as a main-memory-governed process*
- *1 mark: explains Fetch (PC→MAR→memory→MBR→IR, PC increment)*
- *1 mark: identifies Memory/Indirect (data read/write) as the second main-memory-governed process*
- *1 mark: explains the memory data transfer (effective address→MAR, data via MBR)*
- *1 mark: first correct Execute-phase operation listed*
- *1 mark: second correct Execute-phase operation listed*
- *1 mark: explains PC supplies address of next instruction (to MAR/address bus)*
- *1 mark: explains PC is incremented after fetch*
- *1 mark: explains instruction loaded into IR via MBR*
- *1 mark: explains IR holds opcode passed to control unit/decoder*

Model confidence: high.

Model note: The question's phrasing 'TWO processes governed by main memory' is interpreted as the Fetch and Memory/Indirect (data-transfer) steps of the machine cycle, which are the memory-referencing phases. If the lecturer intends 'Fetch and Execute' as the two processes, credit should be given for that interpretation provided the memory role is explained. No figure was supplied; answer is based on the standard von Neumann fetch–decode–execute model.

QUESTION A7 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (8).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Two types of interrupt [4 marks]

1. Maskable interrupt. A maskable interrupt is a hardware interrupt that the CPU can temporarily ignore or delay by setting a mask bit in its interrupt-enable/flag register. It is typically raised by external devices (e.g. a keyboard, timer, or I/O controller) requesting service. Because it can be masked, the processor can disable it during critical sections of code so that time-critical work is not disturbed; it is then serviced when unmasked. Example: an I/O device signalling that a data transfer is complete.

2. Non-maskable interrupt (NMI). A non-maskable interrupt cannot be disabled by the normal interrupt mask and is reserved for high-priority, urgent events such as a power failure, a hardware fault, or a memory parity error. When an NMI is asserted the CPU must respond immediately (after completing the current instruction), transferring control to a dedicated NMI service routine. Because it cannot be masked, it guarantees that catastrophic or critical conditions are handled promptly.

(Other acceptable pairs, if described correctly, include: hardware vs. software interrupt; internal (exception/trap) vs. external interrupt; vectored vs. non-vectored interrupt; edge-triggered vs. level-triggered interrupt.)

(b) Three interfaces to external devices [6 marks]

1. Programmed I/O (polled I/O). The CPU directly controls the transfer and repeatedly checks (polls) a device status register to see whether the device is ready, then reads/writes the data register. It is simple and needs no extra hardware, but wastes CPU time because the processor busy-waits instead of doing useful work.

2. Interrupt-driven I/O. The CPU issues an I/O command and continues with other work; when the device is ready it raises an interrupt, and the CPU runs an interrupt service routine to transfer the data. This frees the CPU from polling and improves efficiency, though each transfer still involves CPU intervention (context save/restore and ISR execution).

3. Direct Memory Access (DMA). A dedicated DMA controller transfers blocks of data directly between the device and main memory without CPU involvement per word. The CPU sets up the transfer (start address, count, direction) and is interrupted only when the whole block is complete, giving high throughput for large transfers such as disk or network I/O.

© Hak Cipta Universiti Teknologi MARA

(Other acceptable interfaces, if explained, include: memory-mapped I/O vs. isolated/port-mapped I/O; serial interfaces such as UART/USB; parallel interfaces; and channel I/O.)

Unofficial marking guide:

• 1 mark: correctly names/identifies a first type of interrupt (e.g. maskable or hardware interrupt)
• 1 mark: describes how the first type works / when it is used
• 1 mark: correctly names/identifies a second type of interrupt (e.g. non-maskable, software/trap, vectored)
• 1 mark: describes how the second type works / when it is used
• 1 mark: correctly identifies and explains the first external-device interface (e.g. programmed/polled I/O)
• 1 mark: states an advantage or limitation of the first interface
• 1 mark: correctly identifies and explains the second interface (e.g. interrupt-driven I/O)
• 1 mark: states an advantage or limitation of the second interface
• 1 mark: correctly identifies and explains the third interface (e.g. DMA or memory-mapped I/O)
• 1 mark: states an advantage or limitation of the third interface

Model confidence: high.

Model note: Question text is clear; no figure or table is required. The marking guide is an unofficial allocation consistent with the 4 + 6 mark split and accepts any two correctly described interrupt types and any three correctly explained device interfaces.

QUESTION A8 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (12).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Registers are small, high-speed storage locations inside the CPU used to hold data, instructions, addresses and intermediate results temporarily during processing. They are the fastest storage in the memory hierarchy and are directly accessible by the CPU's control and arithmetic units. Two types of registers are: (i) the Program Counter (PC), which holds the address of the next instruction to be fetched; and (ii) the Instruction Register (IR), which holds the instruction currently being executed. Other acceptable examples include the Memory Address Register (MAR), Memory Data Register (MDR), Accumulator (AC), and general-purpose registers.

(b) The machine cycle consists of fetch, decode, execute and (where applicable) store/interrupt phases. Fetch: the control unit copies the address in the Program Counter (PC) to the Memory Address Register (MAR); the PC is then incremented to point to the next instruction; the memory is read and the instruction word is placed in the Memory Data Register (MDR) and then transferred to the Instruction Register (IR). Decode: the control unit examines the opcode in the IR, determines the operation to be performed and the operands/addresses involved, and generates the appropriate control signals to carry out the instruction in the execute phase.

(c) A dedicated bus has separate, permanently assigned lines for each function (e.g., separate address and data lines), so all transfers can occur simultaneously and with less control overhead, but it requires more physical lines and is more expensive. A multiplexed bus shares the same lines for different functions (e.g., address and data time-multiplexed), which reduces the number of lines and cost but requires additional control logic and time slots, making transfers slower and more complex.

Unofficial marking guide:

• 1 mark: correct definition of registers as small, fast CPU storage for temporary data/instructions/addresses.
• 1 mark: additional clarification that registers are directly accessible by the CPU and are the fastest storage.
• 1 mark: one correct type of register named (e.g., PC, IR, MAR, MDR, Accumulator).
• 1 mark: a second correct type of register named.
• 1 mark: fetch phase correctly described (PC→MAR, PC incremented, memory read, instruction→MDR→IR).
• 1 mark: fetch phase includes role of PC and memory transfer.
• 1 mark: decode phase correctly described (opcode examined in IR, operation and operands determined).
• 1 mark: decode phase includes generation of control signals for execution.
• 1 mark: correct difference stated for dedicated bus (separate lines per function, simultaneous transfers, more lines/cost).
• 1 mark: correct contrast for multiplexed bus (shared lines, fewer lines, time-multiplexed, slower/more control overhead).

Model confidence: high.

Model note: The question text has a minor OCR inconsistency: the subpart labels show (a), (b), (c) but the slot is labelled A8; the answer addresses all three subparts as listed. No figure or table is required. The marking guide is an unofficial model answer and not an official marking scheme.

© Hak Cipta Universiti Teknologi MARA

QUESTION A9 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (10).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Three types of interfaces to external devices [6 marks]

1. Programmed I/O (polling) interface: The processor directly controls the device through an I/O module and repeatedly checks (polls) the device's status register to see whether it is ready to transfer data. Data are moved between the CPU and the device via the I/O module under direct program control. It is simple and needs no special interrupt hardware, but it wastes CPU time because the processor must busy-wait for slow devices.

2. Interrupt-driven I/O interface: The I/O module signals the processor with an interrupt request when a device is ready or has completed a transfer. The processor suspends its current program, runs an interrupt service routine (ISR) to transfer the data or service the device, then resumes. This frees the CPU from polling, though each transfer still involves CPU intervention and interrupt overhead.

3. Direct Memory Access (DMA) interface: A DMA controller transfers blocks of data directly between the device and main memory without the CPU handling each word. The CPU programs the DMA controller (start address, count, direction) and is interrupted only once, when the whole block transfer is complete. This gives high throughput for fast devices such as disks and network cards.

(Other acceptable interfaces: memory-mapped I/O vs. isolated/port-mapped I/O, serial vs. parallel interfaces, or synchronous vs. asynchronous interfaces, provided each is described.)

(b) Two types of interrupt [4 marks]

1. Maskable interrupt: An interrupt that can be disabled (masked) by the processor, e.g. by clearing an interrupt-enable bit or setting a mask register. It is normally used for ordinary I/O devices whose service can be delayed; the CPU ignores it while masked and services it later when unmasked.

2. Non-maskable interrupt (NMI): An interrupt that cannot be disabled by the normal masking mechanism. It is reserved for high-priority or catastrophic events such as power failure, memory parity error or hardware fault, and is always recognised by the CPU so that urgent action can be taken.

(Alternative acceptable pairing: hardware interrupt vs. software/trap interrupt, or internal (exception) vs. external interrupt, if each is explained.)

Unofficial marking guide:

- *1 mark: naming the first interface type (e.g. programmed I/O/polling)*
- *1 mark: describing how the first interface operates and its CPU involvement/limitation*
- *1 mark: naming the second interface type (e.g. interrupt-driven I/O)*
- *1 mark: describing how the second interface operates (interrupt request, ISR, resume)*
- *1 mark: naming the third interface type (e.g. DMA)*
- *1 mark: describing how the third interface operates (direct memory transfer, single completion interrupt)*
- *1 mark: naming/defining the first interrupt type (e.g. maskable)*
- *1 mark: explaining its behaviour (can be disabled/masked, used for ordinary I/O)*
- *1 mark: naming/defining the second interrupt type (e.g. non-maskable)*
- *1 mark: explaining its behaviour (cannot be masked, used for urgent/fatal events)*

Model confidence: high.

Model note: Question text is complete; no figure or table is required. Alternative valid interface/interrupt pairings are accepted as indicated in the answer, since the paper does not prescribe specific types.

QUESTION A10 (10 marks)

Topic: Data representation and number systems

a) Answer: 100.510 = 0x42C90000 (sign=0, exponent=10000101, fraction=10010010000000000000000)

- *1 mark for sign, exponent and mantissa.*
- *1 mark for the biased exponent.*
- *1 mark for the 23-bit fraction.*
- *1 mark for the 32-bit binary grouping.*
- *1 mark for the hexadecimal result.*

b) Answer: 0x3EC00000 = 0.37510

- *1 mark for the sign bit.*

© Hak Cipta Universiti Teknologi MARA

- *1 mark for the exponent field.*
- *1 mark for the fraction field.*
- *1 mark for applying the bias and significand.*
- *1 mark for the final decimal value.*

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION B1 (20 marks)

Topic: Assembly-language programming

Source: CSC429 (14).pdf, page(s) 6

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

Assumption on the equation: Figure 4 is read as z = A + B² − C(D − C/B), i.e. B2 is taken as B squared (B^2). The expression is evaluated in the order: t1 = C/B; t2 = D − t1; t3 = C × t2; t4 = B × B; t5 = A + t4; z = t5 − t3. Registers A, B, C, D are read-only; x and y are scratch; the result is left in z.

Zero-address program (stack machine). Each operand is pushed, the operation consumes the top two stack entries and pushes the result; POP stores the top of stack into the named register.

```
; z = A + B^2 - C*(D - C/B)
PUSH C      ; stack: C
PUSH B      ; stack: C, B
DIV         ; stack: C/B           (t1)
PUSH D      ; stack: t1, D
SUB         ; stack: D - C/B       (t2)
PUSH C      ; stack: t2, C
MUL         ; stack: C*(D - C/B)   (t3)
PUSH B      ; stack: t3, B
PUSH B      ; stack: t3, B, B
MUL         ; stack: t3, B^2       (t4)
PUSH A      ; stack: t3, t4, A
ADD         ; stack: t3, A + B^2   (t5)
SUB         ; stack: (A + B^2) - t3
POP z       ; z = A + B^2 - C*(D - C/B)
```

One-address program (accumulator machine). The accumulator holds one operand; LOADx copies x into the accumulator, OPx applies the operation between the accumulator and x, and STORx writes the accumulator to x.

```
; z = A + B^2 - C*(D - C/B)
LOAD C      ; ACC = C
DIV B       ; ACC = C/B            (t1)
STOR x      ; x = C/B
LOAD D      ; ACC = D
SUB x       ; ACC = D - C/B        (t2)
STOR x      ; x = D - C/B
LOAD C      ; ACC = C
MUL x       ; ACC = C*(D - C/B)    (t3)
STOR x      ; x = t3
LOAD B      ; ACC = B
MUL B       ; ACC = B^2            (t4)
ADD A       ; ACC = A + B^2        (t5)
SUB x       ; ACC = (A + B^2) - t3
STOR z      ; z = final answer
```

Two-address program (register/memory-register machine). MOVE(x,y) copies y into x; OP(x,y) performs x = x OP y, so the destination register is overwritten and must be chosen carefully because A, B, C, D are read-only.

```
; z = A + B^2 - C*(D - C/B)
MOVE x, C   ; x = C
DIV  x, B   ; x = C/B              (t1)
MOVE y, D   ; y = D
SUB  y, x   ; y = D - C/B          (t2)
MOVE x, C   ; x = C
MUL  x, y   ; x = C*(D - C/B)      (t3)
MOVE y, B   ; y = B
MUL  y, B   ; y = B^2              (t4)
```

© Hak Cipta Universiti Teknologi MARA

```
MOVE z, A   ; z = A
ADD  z, y   ; z = A + B^2        (t5)
SUB  z, x   ; z = (A + B^2) - t3  = final answer
```

In all three versions the final result is held in z, and A, B, C, D are never used as destinations, satisfying the constraints.

Unofficial marking guide:

• 4 marks: zero-address program — correct PUSH/POP/ADD/SUB/MUL/DIV sequence, correct operator order for C/B, D−C/B, C×(…), B×B, A+B², final subtraction, and POP z

• 4 marks: one-address program — correct LOAD/STOR/OPx sequence with accumulator handling and correct use of x as temporary

• 4 marks: two-address program — correct MOVE/OP(x,y) sequence with correct destination choice and no modification of A, B, C, D

• 3 marks: correct evaluation order/parenthesisation of z = A + B² − C(D − C/B) in all three programs (division before subtraction, multiplication before final subtraction)

• 3 marks: correct assembly notation, comments and final result stored in z in each program

• 2 marks: no read-only register (A, B, C, D) is overwritten and only x, y, z are used as temporaries

Model confidence: medium.

Model note: Figure 4 is only available as extracted text; 'B 2' is assumed to mean B squared (B^2). If the original figure shows B×2 instead, replace each B×B / MUL B step with a multiply by constant 2. Table 3 OCR is imperfect ('T\vr.-address' = two-address); instruction mnemonics are interpreted conventionally: zero-address PUSH/POP/ADD/SUB/MUL/DIV, one-address LOADx/STORx/ADDx/SUBx/MULx/DIVx, two-address MOVE(x,y)/ADD(x,y)/SUB(x,y)/MUL(x,y)/DIV(x,y). This is an unofficial model answer, not a marking scheme.

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA